

CPPDescent

9e98e11 (with uncommitted changes)

Generated by Doxygen 1.9.7

1 CPP-Descent	1
1.1 Contributors	1
1.2 Dependencies	2
1.2.1 Minor dependencies	2
1.3 Running the project	2
1.3.1 Results	2
1.4 Documentation	2
1.4.1 Code Structure	2
1.4.2 Code Coverage	2
1.4.3 Profiling	3
1.5 Bibliography	3
2 Namespace Index	5
2.1 Namespace List	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 cppdescent Namespace Reference	11
5.1.1 Detailed Description	12
5.1.2 Function Documentation	12
5.1.2.1 compareEdgesEuclidean()	12
5.1.2.2 compareEdgesManhattan()	12
5.1.2.3 euclideanDistance()	12
5.1.2.4 KNNBruteForceGraph()	13
5.1.2.5 manhattanDistance()	13
5.1.2.6 NNDescent_KNNGraph()	14
5.1.2.7 NNDescent_Query()	14
5.1.2.8 readBinData()	14
5.1.2.9 readBinGraph()	15
5.1.2.10 recall()	15
5.1.2.11 writeBinGraph()	16
6 Class Documentation	17
6.1 Graph Class Reference	17
6.1.1 Member Function Documentation	17
6.1.1.1 getGeneralNeighborsV()	17
6.1.1.2 insertVertex()	18
6.2 GraphVertex Class Reference	18
6.3 GraphVertexPair Class Reference	19

6.4 PQueue Class Reference	19
6.4.1 Detailed Description	20
6.4.2 Constructor & Destructor Documentation	20
6.4.2.1 PQueue()	20
6.4.2.2 ~PQueue()	20
6.4.3 Member Function Documentation	20
6.4.3.1 find()	20
6.4.3.2 getMax()	21
6.4.3.3 getMin()	21
6.4.3.4 getSize()	21
6.4.3.5 insert()	21
6.4.3.6 remove()	22
6.4.3.7 removeMax()	22
6.4.3.8 setDestroyValue()	22
6.4.3.9 toVector()	22
6.5 sets Struct Reference	23
6.6 Vector Class Reference	23
6.6.1 Detailed Description	24
6.6.2 Constructor & Destructor Documentation	24
6.6.2.1 Vector()	24
6.6.2.2 ~Vector()	25
6.6.3 Member Function Documentation	25
6.6.3.1 binaryFind()	25
6.6.3.2 find()	25
6.6.3.3 findNode()	25
6.6.3.4 findPos()	26
6.6.3.5 first()	26
6.6.3.6 getAt()	26
6.6.3.7 getSize()	27
6.6.3.8 insertLast()	27
6.6.3.9 last()	27
6.6.3.10 next()	27
6.6.3.11 nodeValue()	28
6.6.3.12 previous()	28
6.6.3.13 removeLast()	28
6.6.3.14 setAt()	29
6.6.3.15 setDestroyValue()	29
6.7 vectorNode Class Reference	29
6.7.1 Detailed Description	30
6.7.2 Constructor & Destructor Documentation	30
6.7.2.1 vectorNode()	30
6.7.3 Member Function Documentation	30

6.7.3.1 <code>getValue()</code>	30
6.7.3.2 <code>setValue()</code>	30
7 File Documentation	31
7.1 ADTGraph.hpp File Reference	31
7.1.1 Detailed Description	32
7.2 ADTGraph.hpp	33
7.3 ADTPQueue.hpp File Reference	34
7.3.1 Detailed Description	35
7.4 ADTPQueue.hpp	35
7.5 ADTVector.hpp File Reference	35
7.5.1 Detailed Description	37
7.6 ADTVector.hpp	37
7.7 common.hpp	38
7.8 cppdescent.hpp	38
7.9 ADTGraph.cpp File Reference	38
7.9.1 Detailed Description	39
7.10 ADTPQueue.cpp File Reference	40
7.10.1 Detailed Description	40
7.11 ADTVector.cpp File Reference	41
7.11.1 Detailed Description	41
7.12 cppdescent.cpp File Reference	42
7.12.1 Detailed Description	43
7.12.2 Function Documentation	43
7.12.2.1 <code>getSets()</code>	43
7.12.2.2 <code>sampleGraph()</code>	44
Index	45

Chapter 1

CPP-Descent

Part of the “Software Development for Computing Systems” course of the Department of Informatics and Telecommunications, UoA for the first semester of the academic year 2023-2024.

A library that implements the KNN-Graph creation and search algorithms described in [1], [2].

1.1 Contributors

- **Konstantinos Chousos** (1115202000215, sdi2000215 at di.uoa.gr)
 - 1st submission
 - * Build/Testing/Code coverage pipeline
 - * ADTVector, ADTList, ADTPQueue
 - * Brute Force K-NN Graph creation. NN-Descent Graph creation
 - * (Incomplete) Query point
 - 2nd submission
 - * I/O of graphs in binary files
 - * Recall function
 - * Metric functions
 - * Query point
 - * Early termination
 - * Local join
- **Anastasios-Fedon Seitanidis** (1115202000179, sdi2000179 at di.uoa.gr)
 - 1st submission
 - * ADTMap, ADTGraph
 - * Brute Force K-NN Graph creation
 - * NN-Descent Graph creation
 - 2nd submission
 - * Implementation of GraphVertex class
 - * Refactor of previous Graph implementation to use GraphVertex class (without changing the existing interface)
 - * getMin() property to Priority Queue
 - * remove() property to Priority Queue
 - * Implementation of find() helper function for Priority Queue

1.2 Dependencies

For local development, you will need `Make` and `CMake`. To install them on an apt-based linux distro, run the following commands:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install make
$ sudo apt install cmake
```

1.2.1 Minor dependencies

- The project uses [Google Test](#) for its testing needs. Gtest is used as a git submodule of this repo, that is downloaded by CMake the first time you clone. That means that nothing extra needs to be installed to run the tests.
- If you would like to generate the documentation, you will need `doxygen` installed.

1.3 Running the project

To run the main executable, the following commands suffice:

```
$ cmake -S . -B build
$ cmake --build build
$ ./build/app/app <K> <filepath-to-dataset> <dimensions> <1 or 2> <delta>
```

The second from last argument concerns the metric function that will be used. 1 means that the euclidean distance will be used and 2 stands for the manhattan distance.

1.3.1 Results

The *recall* percentage that is printed during execution is computed by the following formula, derived from [1, Sec. 3.2].

The recall of one object is the number of its true K-NN members found divided by K. The recall of an approximate K-NN is the average recall of all objects.

1.4 Documentation

1.4.1 Code Structure

In the [wiki](#), you can find documentation generated by [Doxygen](#). It is also available in [pdf form](#).

1.4.2 Code Coverage

This project uses [LCOV](#) for its code coverage needs. The results are uploaded upon commit [here](#).

1.4.3 Profiling

`Callgrind` and `Qcachegrind` are used for any profiling purposes.

1.5 Bibliography

[1] W. Dong, C. Moses, and K. Li, “Efficient k-nearest neighbor graph construction for generic similarity measures,” in Proceedings of the 20th international conference on World wide web, in WWW '11. New York, NY, USA: Association for Computing Machinery, Mar. 2011, pp. 577–586. doi: 10.1145/1963405.1963487.

[2] “How PyNNDescent works — pynndescent 0.5.0 documentation.” Accessed: Oct. 10, 2023. [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

cppdescent	
Functions for the creation of a K-NN graph	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Graph	17
GraphVertex	18
GraphVertexPair	19
PQueue	
ADT Priority Queue	19
sets	23
Vector	
ADT Vector	23
vectorNode	
Class for the vector Node object	29

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

ADTGraph.hpp		
Abstract undirected graph with weighted edges	...	31
ADTPQueue.hpp	...	34
ADTVector.hpp		
ADTVector implementation using a dynamic array	...	35
common.hpp	...	38
cppdescent.hpp	...	38
ADTGraph.cpp	...	38
ADTPQueue.cpp		
An ADT Priority Queue implemented using a heap	...	40
ADTVector.cpp		
Implementation of ADTVector using a dynamic array	...	41
cppdescent.cpp		
Implementation of the cppdescent library	...	42

Chapter 5

Namespace Documentation

5.1 cppdescent Namespace Reference

Functions for the creation of a K-NN graph.

Functions

- int **compareGraphVertices** (Pointer vertex1, Pointer vertex2)
- int **compareVertices** (Pointer first, Pointer second)
- int **compareNeighbors** (Pointer a, Pointer b)
- int **compareGraphVertexPairs** (Pointer p1, Pointer p2)
- [Vector](#) * **readBinData** (const char *fp, int dimensions)
Reads the data from a binary file.
- void **writeBinGraph** (const char *fp, [Graph](#) *graph, int K)
Writes a computed graph to a binary file.
- [Graph](#) * **readBinGraph** (const char *fp, int dimensions)
Reads a graph from a binary file. To work correctly, the file needs to be first created from the 'writeBinGraph' function.
- float **recall** ([Graph](#) *bfGraph, [Graph](#) *nnGraph, int N, int K)
Returns the recall of the graph computed by NN-Descent, compared to the brute force graph.
- float **euclideanDistance** (Pointer a, Pointer b)
Returns the Euclidean distance between two points of arbitrary dimension.
- float **manhattanDistance** (Pointer a, Pointer b)
Returns the Manhattan distance between two points of arbitrary dimension.
- int **compareEdgesEuclidean** (Pointer first, Pointer second)
Compare edges using the euclideanDistance function.
- int **compareVertexPairsEuclidean** (Pointer first, Pointer second)
- int **compareEdgesManhattan** (Pointer first, Pointer second)
Compare edges using the manhattanDistance function.
- [Graph](#) * **KNNBruteForceGraph** ([Vector](#) *data, int K, CompareFunc compare)
Computes the K-NN graph using brute force.
- [Graph](#) * **NNDescent_KNNGraph** ([Vector](#) *data, int K, float delta=0.01, float rho=0.5, DistanceFunc distance=**euclideanDistance**)
Computes the K-NN graph for the given dataset using the NN-Descent algorithm.
- [PQueue](#) * **NNDescent_Query** ([Graph](#) *graph, int K, CompareFunc compare, [Vector](#) *query)
Computes the K Nearest Neighbors of the query point in the graph.

5.1.1 Detailed Description

Functions for the creation of a K-NN graph.

5.1.2 Function Documentation

5.1.2.1 `compareEdgesEuclidean()`

```
int cppdescent::compareEdgesEuclidean (
    Pointer first,
    Pointer second )
```

Compare edges using the `euclideanDistance` function.

Parameters

<i>first</i>	A Pointer to the first element.
<i>second</i>	A Pointer to the second element.

Returns

int

5.1.2.2 `compareEdgesManhattan()`

```
int cppdescent::compareEdgesManhattan (
    Pointer first,
    Pointer second )
```

Compare edges using the `manhattanDistance` function.

Parameters

<i>first</i>	A Pointer to the first element.
<i>second</i>	A Pointer to the second element.

Returns

int

5.1.2.3 `euclideanDistance()`

```
float cppdescent::euclideanDistance (
    Pointer a,
    Pointer b )
```

Returns the Euclidean distance between two points of arbitrary dimension.

Parameters

<i>first</i>	A pointer to the first point.
<i>second</i>	A pointer to the second point.

Returns

long double The Euclidean distance.

5.1.2.4 KNNBruteForceGraph()

```
Graph * cppdescent::KNNBruteForceGraph (
    Vector * data,
    int K,
    CompareFunc compare )
```

Computes the K-NN graph using brute force.

Useful for testing the recall and the performance of our NN-Descent algorithm.

The user is responsible deletion of the allocated memory of the graph.

Parameters

<i>data</i>	A pointer to the parent N-sized vector.
<i>K</i>	The number of Nearest Neighbors to find.
<i>compare</i>	The function to use to compare the distances.

Returns

Graph* A pointer to the optimal K-NN graph.

5.1.2.5 manhattanDistance()

```
float cppdescent::manhattanDistance (
    Pointer a,
    Pointer b )
```

Returns the Manhattan distance between two points of arbitrary dimension.

Parameters

<i>first</i>	A pointer to the first point.
<i>second</i>	A pointer to the second point.

Returns

long double The Manhattan distance.

5.1.2.6 NNDescent_KNNGraph()

```
Graph * cppdescent::NNDescent_KNNGraph (
    Vector * data,
    int K,
    float delta = 0.01,
    float rho = 0.5,
    DistanceFunc distance = euclideanDistance )
```

Computes the K-NN graph for the given dataset using the NN-Descent algorithm.

The user is responsible deletion of the allocated memory of the returned graph.

Parameters

<i>data</i>	A Vector of the vertices of the graph.
<i>K</i>	The number of nearest neighbors to compute.
<i>delta</i>	The iterations will stop when the number of edges that were updated is less than $\text{delta} * N * K$.
<i>rho</i>	The sampling rate
<i>distance</i>	The function to be used to compute the distances between vertices.

Returns

Graph* The complete K-NN graph of the dataset.

5.1.2.7 NNDescent_Query()

```
PQueue * cppdescent::NNDescent_Query (
    Graph * graph,
    int K,
    CompareFunc compare,
    Vector * query )
```

Computes the K Nearest Neighbors of the query point in the graph.

Parameters

<i>graph</i>	The K-NN Graph , in which we search for neighbors of the query.
<i>K</i>	
<i>compare</i>	The function used to compare edges.
<i>query</i>	The query point, given as a Vector . Must be of the same dimensions as the rest points of the graph.

Returns

PQueue* A priority queue of the K-NN of the query, with 'max' being the most distant point relative to the query. It contains GraphVertexPairs from the query point directed to the other nodes. If the query point is of wrong dimensions, then nullptr is returned.

5.1.2.8 readBinData()

```
Vector * cppdescent::readBinData (
```

```
const char * fp,
int dimensions )
```

Reads the data from a binary file.

The data read are stored in a N-sized [Vector](#), where N is the `<uint32_t>` at the start of the file. Each node of this vector is a pointer to another 100-sized vector, of which the values are pointers to floats.

All of the vectors (the parent N-sized and each of the 100-sized) must be freed by the user.

Parameters

<i>fp</i>	A string (char *) of the filepath to the binary dataset.
<i>dimensions</i>	The dimension of each point.

Returns

Vector* An N-sized vector with vectors for each element.

5.1.2.9 readBinGraph()

```
Graph * cppdescent::readBinGraph (
    const char * fp,
    int dimensions )
```

Reads a graph from a binary file. To work correctly, the file needs to be first created from the 'writeBinGraph' function.

Parameters

<i>fp</i>	The filepath.
<i>dimensions</i>	The dimensions of the datapoints.

Returns

Graph*

5.1.2.10 recall()

```
float cppdescent::recall (
    Graph * bfGraph,
    Graph * nnGraph,
    int N,
    int K )
```

Returns the recall of the graph computed by NN-Descent, compared to the brute force graph.

Parameters

<i>bfGraph</i>	
<i>nnGraph</i>	
<i>K</i>	

Returns

float

5.1.2.11 writeBinGraph()

```
void cppdescent::writeBinGraph (
    const char * fp,
    Graph * graph,
    int K )
```

Writes a computed graph to a binary file.

The structure of the binary file that will be created is the following:

1. <uint32_t> N: the number of the vertices
2. N * 100 floats: each vertex of 100 dimensions
3. N * K * 1 int: For each vertex sequentially, the positions of its K neighbors.

Parameters

<i>fp</i>	The filepath to the created file.
<i>K</i>	
<i>graph</i>	

Chapter 6

Class Documentation

6.1 Graph Class Reference

Public Member Functions

- **Graph** (CompareFunc compare, DestroyFunc vecDestroy)
- int **getSize** ()
- void **insertVertex** (Pointer vertex)
Insert a vertex to the graph (if it doesn't already exist).
- **Vector** * **getVerticesV** ()
- void **removeVertex** (Pointer vertex)
- void **insertEdge** (Pointer vertex1, Pointer vertex2)
- void **removeEdge** (Pointer vertex1, Pointer vertex2)
- **Vector** * **getAdjacentV** (Pointer vertex)
- **Vector** * **getReverseAdjacentV** (Pointer vertex)
- **Vector** * **getGeneralNeighborsV** (Pointer vertex)
Returns a vector where the K first elements are the direct neighbors of the vertex and the rest are the reverse ones.
- bool **isNeighborVertex** (Pointer v1, Pointer v2)
- CompareFunc **getCompareData** ()
- CompareFunc **getCompareVertices** ()
- DestroyFunc **getDestroyData** ()
- **Vector** * **getVec** ()

6.1.1 Member Function Documentation

6.1.1.1 getGeneralNeighborsV()

```
Vector * Graph::getGeneralNeighborsV (  
    Pointer vertex )
```

Returns a vector where the K first elements are the direct neighbors of the vertex and the rest are the reverse ones.

Parameters

<i>vertex</i>	
---------------	--

Returns

Vector*

6.1.1.2 insertVertex()

```
void Graph::insertVertex (
    Pointer vertex )
```

Insert a vertex to the graph (if it doesn't already exist).

Parameters

<i>vertex</i>	A Pointer to the vertex to be unserted.
---------------	---

The documentation for this class was generated from the following files:

- [ADTGraph.hpp](#)
- [ADTGraph.cpp](#)

6.2 GraphVertex Class Reference

Public Member Functions

- **GraphVertex** (Pointer data, [Graph](#) *owner)
- void **addNeighbor** (Pointer neighbor)
- void **addReverse** (Pointer reverse)
- void **removeNeighbor** (Pointer neighbor, CompareFunc compare)
- void **removeReverse** (Pointer reverse, CompareFunc compare)
- [PQueue](#) * **getNeighbors** ()
- [PQueue](#) * **getReverse** ()
- Pointer **getData** ()
- [Graph](#) * **getOwner** ()
- void **check** ()
- bool **checked** ()
- void **setNorm** (double norm)
- double **getNorm** ()

The documentation for this class was generated from the following files:

- [ADTGraph.hpp](#)
- [ADTGraph.cpp](#)

6.3 GraphVertexPair Class Reference

Public Member Functions

- **GraphVertexPair** ([Graph](#) *owner, Pointer vertex1, Pointer vertex2)
- Pointer **getVertex1** ()
- Pointer **getVertex2** ()
- [Graph](#) * **getOwner** ()
- void **setFalse** ()
- bool **getFlag** ()

The documentation for this class was generated from the following file:

- [ADTGraph.hpp](#)

6.4 PQueue Class Reference

ADT Priority Queue.

```
#include <ADTPQueue.hpp>
```

Public Member Functions

- [PQueue](#) (CompareFunc compare, DestroyFunc destroyValue, [Vector](#) *values)
Construct a new [PQueue](#) object.
- [~PQueue](#) ()
Destroy the [PQueue](#) object.
- int [getSize](#) ()
Get the size of the priority queue.
- Pointer [getMax](#) ()
Get the max element of the queue.
- Pointer [getMin](#) ()
Get the min element of the queue.
- void [insert](#) (Pointer value)
Insert a new element to the queue.
- void [removeMax](#) ()
Remove the max of the queue.
- DestroyFunc [setDestroyValue](#) (DestroyFunc destroyValue)
Set the Destroy Value object.
- void [remove](#) (Pointer value, CompareFunc compare)
Finds (using the given compare function) and removes the node with the given value.
- int [find](#) (Pointer value, CompareFunc compare)
Finds the given value in the priority queue and returns its nodeId.
- [Vector](#) * [toVector](#) ()
Returns all the elements as a vector.
- Pointer **nodeValue** (int nodeId)
- void **nodeSwap** (int nodeId1, int nodeId2)
- void **bubbleUp** (int nodeId)
- void **bubbleDown** (int nodeId)
- void **naiveHeapify** ([Vector](#) *values)

6.4.1 Detailed Description

ADT Priority Queue.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 PQueue()

```
PQueue::PQueue (
    CompareFunc compare,
    DestroyFunc destroyValue,
    Vector * values )
```

Construct a new [PQueue](#) object.

Uses the `compare` function to compare its elements.

If `destroyValue != nullptr`, the `destroyValue` function is called upon removal of an element.

If `values != nullptr`, then the priority queue will be initialized with the values of the vector `values`.

Parameters

<i>compare</i>	
<i>destroyValue</i>	
<i>values</i>	

6.4.2.2 ~PQueue()

```
PQueue::~~PQueue ( )
```

Destroy the [PQueue](#) object.

6.4.3 Member Function Documentation

6.4.3.1 find()

```
int PQueue::find (
    Pointer value,
    CompareFunc compare )
```

Finds the given value in the priority queue and returns its nodeId.

Parameters

<i>value</i>	The value we want to find.
<i>compare</i>	Compare function to use.

Returns

int

6.4.3.2 getMax()

```
Pointer PQueue::getMax ( )
```

Get the max element of the queue.

Returns

Pointer A generic pointer to the max element.

6.4.3.3 getMin()

```
Pointer PQueue::getMin ( )
```

Get the min element of the queue.

Returns

Pointer A generic pointer to the min element.

6.4.3.4 getSize()

```
int PQueue::getSize ( )
```

Get the size of the priority queue.

Returns

int

6.4.3.5 insert()

```
void PQueue::insert (
    Pointer value )
```

Insert a new element to the queue.

Parameters

<i>value</i>	The value of the new element to be inserted.
--------------	--

6.4.3.6 remove()

```
void PQueue::remove (
    Pointer value,
    CompareFunc compare )
```

Finds (using the given compare function) and removes the node with the given value.

Parameters

<i>value</i>	The value we want to remove.
<i>compare</i>	Compare function to use for finding the node we want to remove.

6.4.3.7 removeMax()

```
void PQueue::removeMax ( )
```

Remove the max of the queue.

6.4.3.8 setDestroyValue()

```
DestroyFunc PQueue::setDestroyValue (
    DestroyFunc destroyValue )
```

Set the Destroy Value object.

Parameters

<i>destroyValue</i>	
---------------------	--

Returns

DestroyFunc

6.4.3.9 toVector()

```
Vector * PQueue::toVector ( ) [inline]
```

Returns all the elements as a vector.

Returns

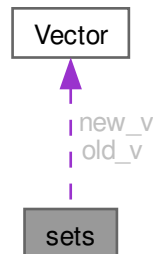
Vector*

The documentation for this class was generated from the following files:

- [ADTPQueue.hpp](#)
- [ADTPQueue.cpp](#)

6.5 sets Struct Reference

Collaboration diagram for sets:



Public Attributes

- [Vector](#) * **new_v**
- [Vector](#) * **old_v**

The documentation for this struct was generated from the following file:

- [cppdescent.cpp](#)

6.6 Vector Class Reference

ADT [Vector](#).

```
#include <ADTVector.hpp>
```

Public Member Functions

- [Vector](#) (int size, DestroyFunc destroyValue)
Construct a new [Vector](#) object.
- [~Vector](#) ()
Destroy the [Vector](#) object.
- int [getSize](#) ()
Get the current size of the [Vector](#).
- void [insertLast](#) (Pointer value)
Inserts a new element at the end of the vector.
- int [removeLast](#) ()
Removes the last element of the vector.
- Pointer [getAt](#) (int pos)
Get the value at the specified position of the vector.

- int [setAt](#) (int pos, Pointer value)
Set the value at the specified position of the vector.
- Pointer [find](#) (Pointer value, CompareFunc compare)
Find the first element with value equal to value.
- Pointer [binaryFind](#) (Pointer value, CompareFunc compare)
Find the element with value equal to value using binary search.
- int [findPos](#) (Pointer value, CompareFunc compare)
Find the first element with value equal to value and return its pos.
- DestroyFunc [setDestroyValue](#) (DestroyFunc destroyValue)
Set the Destroy Value.
- [vectorNode](#) * [first](#) ()
Get the first node of the vector.
- [vectorNode](#) * [last](#) ()
Get the last node of the vector.
- [vectorNode](#) * [next](#) ([vectorNode](#) *node)
Get the next node of the vector, after the one given.
- [vectorNode](#) * [previous](#) ([vectorNode](#) *node)
Get the previous node of the vector, before the one given.
- Pointer [nodeValue](#) ([vectorNode](#) *node)
Get the value of the node.
- [vectorNode](#) * [findNode](#) (Pointer value, CompareFunc compare)
Find the first node with value equal to value.

6.6.1 Detailed Description

ADT [Vector](#).

An ADT [Vector](#) using a dynamic array for smart resizing.

6.6.2 Constructor & Destructor Documentation

6.6.2.1 Vector()

```
Vector::Vector (
    int size,
    DestroyFunc destroyValue )
```

Construct a new [Vector](#) object.

The newly created [Vector](#) will be of size size and its elements will be initialized as nullptr.

If destroyValue is not nullptr, it will be called on each element when it is removed from the [Vector](#).

Parameters

<i>size</i>	The initial size of the Vector .
<i>destroyValue</i>	The function to be called on each element when it is removed from the Vector .

6.6.2.2 ~Vector()

```
Vector::~~Vector ( )
```

Destroy the [Vector](#) object.

Deletes all allocated memory of the [Vector](#).

6.6.3 Member Function Documentation

6.6.3.1 binaryFind()

```
Pointer Vector::binaryFind (
    Pointer value,
    CompareFunc compare )
```

Find the element with value equal to value using binary search.

Parameters

<i>value</i>	The value to look for.
<i>compare</i>	The function to be used for comparison.

Returns

Pointer A pointer to the found value.

6.6.3.2 find()

```
Pointer Vector::find (
    Pointer value,
    CompareFunc compare )
```

Find the first element with value equal to value.

Parameters

<i>value</i>	The value to look for.
<i>compare</i>	The function to be used for comparison.

Returns

Pointer A pointer to the found value.

6.6.3.3 findNode()

```
vectorNode * Vector::findNode (
    Pointer value,
    CompareFunc compare )
```

Find the first node with value equal to value.

Parameters

<i>value</i>	The value to look for.
<i>compare</i>	The function to be used for comparison.

Returns

[vectorNode](#) The resulting node.

6.6.3.4 findPos()

```
int Vector::findPos (
    Pointer value,
    CompareFunc compare )
```

Find the first element with value equal to value and return its pos.

Parameters

<i>value</i>	The value to look for.
<i>compare</i>	The function to be used for comparison.

Returns

int The position of the element.

6.6.3.5 first()

```
vectorNode * Vector::first ( )
```

Get the first node of the vector.

Returns

[vectorNode](#)

6.6.3.6 getAt()

```
Pointer Vector::getAt (
    int pos )
```

Get the value at the specified position of the vector.

Parameters

<i>pos</i>	The position to look for.
------------	---------------------------

Returns

Pointer A pointer to the value at the specified position.

6.6.3.7 getSize()

```
int Vector::getSize ( )
```

Get the current size of the [Vector](#).

Returns

int The size of the [Vector](#).

6.6.3.8 insertLast()

```
void Vector::insertLast (
    Pointer value )
```

Inserts a new element at the end of the vector.

Parameters

<i>value</i>	The value of the new element.
--------------	-------------------------------

If we are at the last element, we create a new array double the size.

6.6.3.9 last()

```
vectorNode * Vector::last ( )
```

Get the last node of the vector.

Returns

[vectorNode](#)

6.6.3.10 next()

```
vectorNode * Vector::next (
    vectorNode * node )
```

Get the next node of the vector, after the one given.

Parameters

<i>node</i>	The node to get the next of.
-------------	------------------------------

Returns

[vectorNode](#)

6.6.3.11 nodeValue()

```
Pointer Vector::nodeValue (
    vectorNode * node )
```

Get the value of the node.

Parameters

<i>node</i>	
-------------	--

Returns

Pointer

6.6.3.12 previous()

```
vectorNode * Vector::previous (
    vectorNode * node )
```

Get the previous node of the vector, before the one given.

Parameters

<i>node</i>	The node to get the previous of.
-------------	----------------------------------

Returns

[vectorNode](#)

6.6.3.13 removeLast()

```
int Vector::removeLast ( )
```

Removes the last element of the vector.

Returns

int -1 if the function fails, else 0.

6.6.3.14 setAt()

```
int Vector::setAt (
    int pos,
    Pointer value )
```

Set the value at the specified position of the vector.

Parameters

<i>pos</i>	The position to edit.
<i>value</i>	The new value.

Returns

int -1 if the function fails, else 0.

6.6.3.15 setDestroyValue()

```
DestroyFunc Vector::setDestroyValue (
    DestroyFunc destroyValue )
```

Set the Destroy Value.

Parameters

<i>destroyValue</i>	The new destroy value function.
---------------------	---------------------------------

Returns

DestroyFunc The old destroy value function.

The documentation for this class was generated from the following files:

- [ADTVector.hpp](#)
- [ADTVector.cpp](#)

6.7 vectorNode Class Reference

Class for the vector Node object.

```
#include <ADTVector.hpp>
```

Public Member Functions

- [vectorNode](#) ()
Construct a new vector Node object.
- **vectorNode** (Pointer value)
- Pointer [getValue](#) () const
Get the Value object.
- void [setValue](#) (Pointer value)
Set the Value object.

6.7.1 Detailed Description

Class for the vector Node object.

A vector node is described only its value.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 vectorNode()

```
vectorNode::vectorNode ( ) [inline]
```

Construct a new vector Node object.

Parameters

<i>value</i>	A Pointer to the value.
--------------	-------------------------

6.7.3 Member Function Documentation

6.7.3.1 getValue()

```
Pointer vectorNode::getValue ( ) const [inline]
```

Get the Value object.

Returns

Pointer

6.7.3.2 setValue()

```
void vectorNode::setValue (
    Pointer value ) [inline]
```

Set the Value object.

Parameters

<i>value</i>	
--------------	--

The documentation for this class was generated from the following file:

- [ADTVector.hpp](#)

Chapter 7

File Documentation

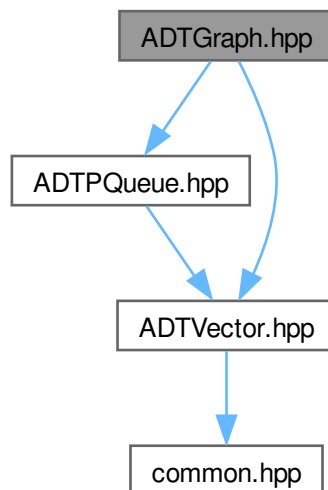
7.1 ADTGraph.hpp File Reference

Abstract undirected graph with weighted edges.

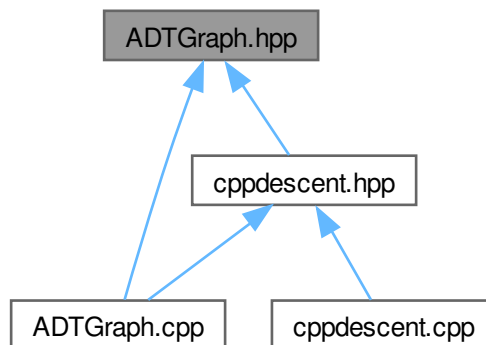
```
#include "ADTPQueue.hpp"
```

```
#include "ADTVector.hpp"
```

Include dependency graph for ADTGraph.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Graph](#)
- class [GraphVertex](#)
- class [GraphVertexPair](#)

7.1.1 Detailed Description

Abstract undirected graph with weighted edges.

Author

Phaedon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

Copyright (c) 2023

7.2 ADTGraph.hpp

[Go to the documentation of this file.](#)

```

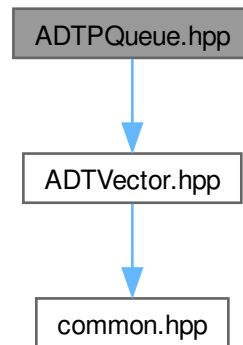
00001
00012 #pragma once
00013
00014 #include "ADTPQueue.hpp"
00015 #include "ADTVector.hpp"
00016
00017 class Graph {
00018 private:
00019     Vector* vec;
00020     int size;
00021     CompareFunc compare_vertices;
00022     CompareFunc compare_data;
00023     DestroyFunc destroy_data;
00024
00025 public:
00026     Graph(CompareFunc compare, DestroyFunc vecDestroy);
00027     ~Graph();
00028     int getSize();
00029     void insertVertex(Pointer vertex);
00030     Vector* getVerticesV();
00031     void removeVertex(Pointer vertex);
00032     void insertEdge(Pointer vertex1, Pointer vertex2);
00033     void removeEdge(Pointer vertex1, Pointer vertex2);
00034     Vector* getAdjacentV(Pointer vertex);
00035     Vector* getReverseAdjacentV(Pointer vertex);
00036     Vector* getGeneralNeighborsV(Pointer vertex);
00037     bool isNeighborVertex(Pointer v1, Pointer v2);
00038     CompareFunc getCompareData() { return this->compare_data; };
00039     CompareFunc getCompareVertices() { return this->compare_vertices; };
00040     DestroyFunc getDestroyData() { return this->destroy_data; };
00041     Vector* getVec() { return this->vec; };
00042 };
00043
00044 class GraphVertex {
00045 private:
00046     Pointer data;
00047     PQueue* neighbors;
00048     PQueue* reverse;
00049     Graph* owner;
00050     bool hasBeenChecked;
00051     double norm;
00052
00053 public:
00054     GraphVertex(Pointer data, Graph* owner);
00055     ~GraphVertex();
00056     void addNeighbor(Pointer neighbor) { this->neighbors->insert(neighbor); };
00057     void addReverse(Pointer reverse) { this->reverse->insert(reverse); };
00058     void removeNeighbor(Pointer neighbor, CompareFunc compare) {
00059         this->neighbors->remove(neighbor, compare);
00060     };
00061     void removeReverse(Pointer reverse, CompareFunc compare) {
00062         this->reverse->remove(reverse, compare);
00063     };
00064     PQueue* getNeighbors() { return this->neighbors; };
00065     PQueue* getReverse() { return this->reverse; };
00066     Pointer getData() { return this->data; };
00067     Graph* getOwner() { return this->owner; };
00068     void check() { this->hasBeenChecked = true; };
00069     bool checked() { return this->hasBeenChecked; };
00070     void setNorm(double norm) { this->norm = norm; };
00071     double getNorm() { return this->norm; };
00072 };
00073
00074 class GraphVertexPair {
00075 public:
00076     GraphVertexPair(Graph* owner, Pointer vertex1, Pointer vertex2)
00077         : flag(true), vertex1(vertex1), vertex2(vertex2), owner(owner){};
00078     Pointer getVertex1() { return this->vertex1; };
00079     Pointer getVertex2() { return this->vertex2; };
00080     Graph* getOwner() { return this->owner; };
00081     void setFalse() { this->flag = false; };
00082     bool getFlag() { return this->flag; };
00083
00084 private:
00085     bool flag;
00086     Pointer vertex1;
00087     Pointer vertex2;
00088     Graph* owner;
00089 };

```

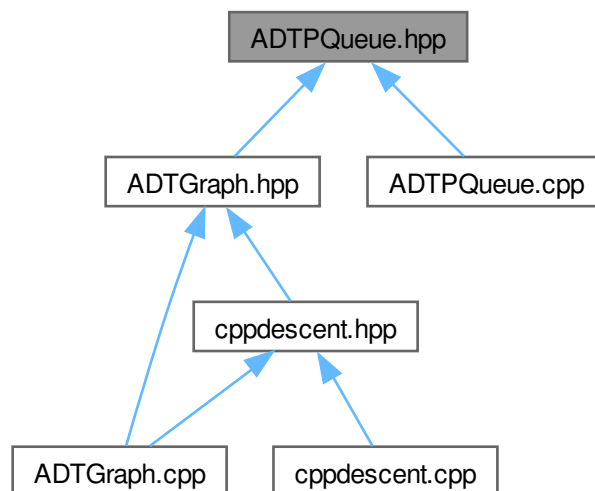
7.3 ADTPQueue.hpp File Reference

```
#include "ADTVector.hpp"
```

Include dependency graph for ADTPQueue.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [PQueue](#)
ADT Priority Queue.

7.3.1 Detailed Description

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

Copyright (c) 2023

7.4 ADTPQueue.hpp

[Go to the documentation of this file.](#)

```

00001
00011 #pragma once
00012
00013 #include "ADTVector.hpp"
00014
00019 class PQueue {
00020 public:
00036     PQueue(CompareFunc compare, DestroyFunc destroyValue, Vector* values);
00041     ~PQueue();
00047     int getSize();
00053     Pointer getMax();
00059     Pointer getMin();
00065     void insert(Pointer value);
00070     void removeMax();
00077     DestroyFunc setDestroyValue(DestroyFunc destroyValue);
00086     void remove(Pointer value, CompareFunc compare);
00094     int find(Pointer value, CompareFunc compare);
00095
00101     Vector* toVector() { return this->vector; };
00102
00103     // Helper functions
00104     // These are used because the node IDs are 1-based, where as the
00105     // vector is 0-based.
00106     Pointer nodeValue(int nodeId);
00107     void nodeSwap(int nodeId1, int nodeId2);
00108     void bubbleUp(int nodeId);
00109     void bubbleDown(int nodeId);
00110     void naiveHeapify(Vector* values);
00111
00112 private:
00113     Vector* vector;
00114     CompareFunc compare;
00115     DestroyFunc destroyValue;
00116 };

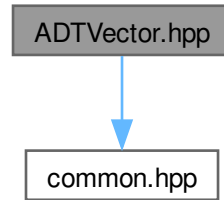
```

7.5 ADTVector.hpp File Reference

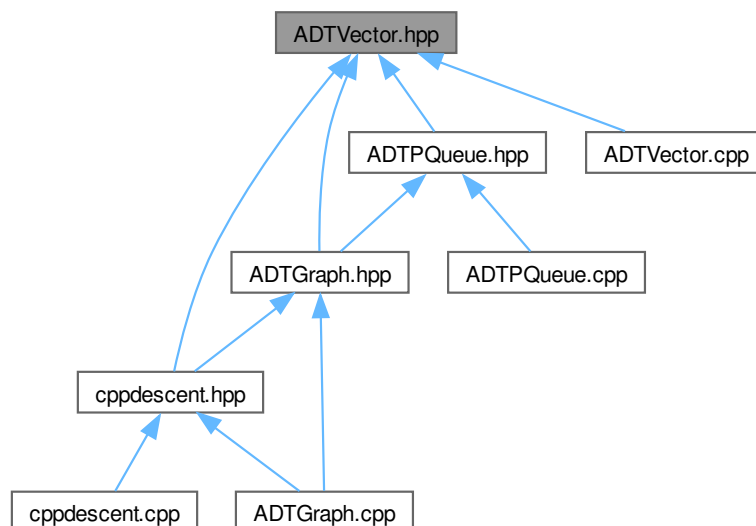
ADTVector implementation using a dynamic array.

```
#include "common.hpp"
```

Include dependency graph for ADTVector.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [vectorNode](#)
Class for the vector Node object.
- class [Vector](#)
ADT [Vector](#).

Macros

- `#define VECTOR_MIN_CAPACITY 10`
- `#define VECTOR_BOF (vectorNode\*)0`
- `#define VECTOR_EOF (vectorNode\*)0`

7.5.1 Detailed Description

ADTVector implementation using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

Copyright

Copyright (c) 2023

7.6 ADTVector.hpp

[Go to the documentation of this file.](#)

```
00001
00011 #pragma once
00012
00013 #include "common.hpp"
00014
00015 #define VECTOR_MIN_CAPACITY 10
00016
00017 #define VECTOR_BOF (vectorNode*)0
00018 #define VECTOR_EOF (vectorNode*)0
00019
00026 class vectorNode {
00027 public:
00033     vectorNode(){};
00034     vectorNode(Pointer value) : value(value){};
00040     Pointer getValue() const { return value; };
00046     void setValue(Pointer value) { this->value = value; };
00047
00048 private:
00049     Pointer value;
00050 };
00051
00058 class Vector {
00059 public:
00073     Vector(int size, DestroyFunc destroyValue);
00080     ~Vector();
00086     int getSize();
00092     void insertLast(Pointer value);
00098     int removeLast();
00105     Pointer getAt(int pos);
00113     int setAt(int pos, Pointer value);
00121     Pointer find(Pointer value, CompareFunc compare);
00129     Pointer binaryFind(Pointer value, CompareFunc compare);
00137     int findPos(Pointer value, CompareFunc compare);
00144     DestroyFunc setDestroyValue(DestroyFunc destroyValue);
00150     vectorNode* first();
00156     vectorNode* last();
00163     vectorNode* next(vectorNode* node);
00170     vectorNode* previous(vectorNode* node);
00177     Pointer nodeValue(vectorNode* node);
00185     vectorNode* findNode(Pointer value, CompareFunc compare);
00186
00187 private:
00188     vectorNode* array;
00189     int size;
00190     int capacity;
00191     DestroyFunc destroyValue;
00192 };
```

7.7 common.hpp

```

00001 #pragma once
00002
00003 typedef void* Pointer;
00004
00005 typedef int (*CompareFunc)(Pointer a, Pointer b);
00006
00007 typedef void (*DestroyFunc)(Pointer value);
00008
00009 typedef unsigned int (*HashFunc)(Pointer);

```

7.8 cppdescent.hpp

```

00001
00005 #include "ADTGraph.hpp"
00006 #include "ADTVector.hpp"
00007
00008 typedef float (*DistanceFunc)(Pointer a, Pointer b);
00009 extern bool verbose;
00010
00015 namespace cppdescent {
00016 int compareGraphVertices(Pointer vertex1, Pointer vertex2);
00017
00018 int compareVertices(Pointer first, Pointer second);
00019
00020 int compareNeighbors(Pointer a, Pointer b);
00021
00022 int compareGraphVertexPairs(Pointer p1, Pointer p2);
00023
00024 // ===== I/O =====
00039 Vector* readBinData(const char* fp, int dimensions);
00040
00057 void writeBinGraph(const char* fp, Graph* graph, int K);
00058
00067 Graph* readBinGraph(const char* fp, int dimensions);
00068
00069 // ===== Helper Functions =====
00080 float recall(Graph* bfGraph, Graph* nnGraph, int N, int K);
00081
00082 // ===== Metric Functions =====
00091 float euclideanDistance(Pointer a, Pointer b);
00100 float manhattanDistance(Pointer a, Pointer b);
00108 int compareEdgesEuclidean(Pointer first, Pointer second);
00109
00110 int compareVertexPairsEuclidean(Pointer first, Pointer second);
00118 int compareEdgesManhattan(Pointer first, Pointer second);
00119
00120 // ===== KNN computation =====
00134 Graph* KNNBruteForceGraph(Vector* data, int K, CompareFunc compare);
00151 Graph* NNDescent_KNNGraph(Vector* data,
00152                             int K,
00153                             float delta = 0.01,
00154                             float rho = 0.5,
00155                             DistanceFunc distance = euclideanDistance);
00169 PQueue* NNDescent_Query(Graph* graph,
00170                           int K,
00171                           CompareFunc compare,
00172                           Vector* query);
00173 }; // namespace cppdescent

```

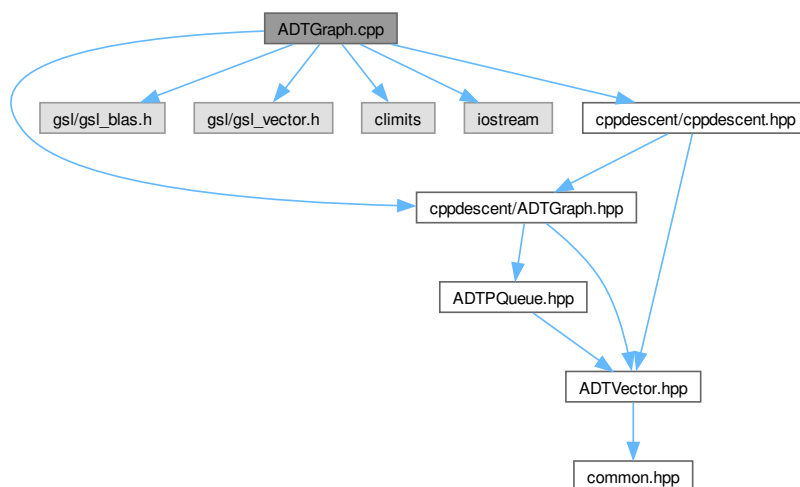
7.9 ADTGraph.cpp File Reference

```

#include "cppdescent/ADTGraph.hpp"
#include <gsl/gsl_blas.h>
#include <gsl/gsl_vector.h>
#include <climits>
#include <iostream>

```

```
#include "cppdescent/cppdescent.hpp"
Include dependency graph for ADTGraph.cpp:
```



Functions

- int **compareVertices** (Pointer vertex1, Pointer vertex2)
- int **compareNeighbors** (Pointer neighbor1, Pointer neighbor2)
- void **destroyVertex** ([GraphVertex](#) *vertex)
- void **destroyEdgePair** ([GraphVertexPair](#) *pair)

7.9.1 Detailed Description

Author

Pheadon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

Copyright (c) 2023

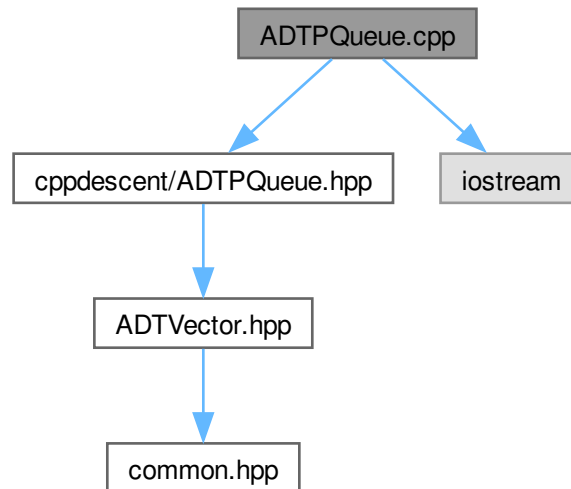
7.10 ADTPQueue.cpp File Reference

An ADT Priority Queue implemented using a heap.

```
#include "cppdescent/ADTPQueue.hpp"
```

```
#include <iostream>
```

Include dependency graph for ADTPQueue.cpp:



7.10.1 Detailed Description

An ADT Priority Queue implemented using a heap.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

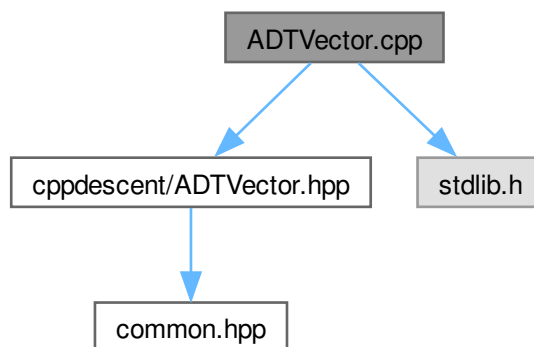
Copyright (c) 2023

7.11 ADTVector.cpp File Reference

Implementation of ADTVector using a dynamic array.

```
#include "cppdescent/ADTVector.hpp"  
#include "stdlib.h"
```

Include dependency graph for ADTVector.cpp:



7.11.1 Detailed Description

Implementation of ADTVector using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

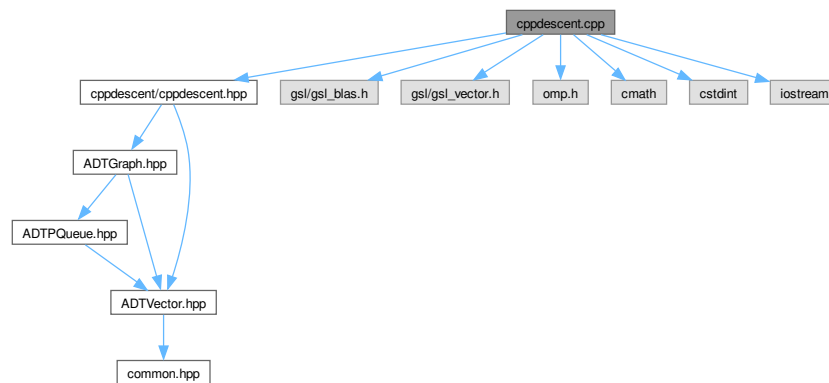
Copyright (c) 2023

7.12 cppdescent.cpp File Reference

Implementation of the cppdescent library.

```
#include "cppdescent/cppdescent.hpp"
#include <gsl/gsl_blas.h>
#include <gsl/gsl_vector.h>
#include <omp.h>
#include <cmath>
#include <stdint>
#include <iostream>
```

Include dependency graph for cppdescent.cpp:



Classes

- struct [sets](#)

Functions

- void **destroyEdges** ([GraphVertexPair](#) *pair)
- [Graph](#) * **sampleGraph** ([Vector](#) *data, int K)
Creates a random graph, where each vertex has K random neighbors.
- int **updateNN** ([Graph](#) *graph, Pointer u1, Pointer u2, float dist, DistanceFunc distance)
- struct [sets](#) **getSets** ([Vector](#) *neighbors, int K, float rho)
Get the Sets object.

Variables

- bool **verbose** = true

7.12.1 Detailed Description

Implementation of the cppdescent library.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-27

Copyright

Copyright (c) 2023

7.12.2 Function Documentation

7.12.2.1 getSets()

```
struct sets getSets (
    Vector * neighbors,
    int K,
    float rho )
```

Get the Sets object.

Returns a `sets` struct containing a vector pointer to the new[v] set and another to the old[v] set.

The first contains $\rho * K$ of direct neighbors with their flag equal to true and $\rho * K$ reverse neighbors with true. In other words, it contains $2 * \rho * K$ neighbors with flag = true.

The second contains all of the direct neighbors with flag = false, which in the worst case will be K , and $\rho * K$ of the reverse neighbors with flag = false. In other words, $K + \rho * K$ neighbors with flag = false.

Parameters

<i>neighbors</i>	
<i>K</i>	
<i>rho</i>	

Returns

struct sets

7.12.2.2 sampleGraph()

```
Graph * sampleGraph (
    Vector * data,
    int K )
```

Creates a random graph, where each vertex has K random neighbors.

The user is responsible for deallocating the graph.

Parameters

<i>data</i>	The Vector with all the points.
<i>K</i>	
<i>compare</i>	
<i>distance</i>	

Returns

Graph* The created graph.

Index

- ~PQueue
 - PQueue, [20](#)
- ~Vector
 - Vector, [24](#)
- ADTGraph.cpp, [38](#)
- ADTGraph.hpp, [31](#), [33](#)
- ADTPQueue.cpp, [40](#)
- ADTPQueue.hpp, [34](#), [35](#)
- ADTVector.cpp, [41](#)
- ADTVector.hpp, [35](#), [37](#)
- binaryFind
 - Vector, [25](#)
- common.hpp, [38](#)
- compareEdgesEuclidean
 - cppdescent, [12](#)
- compareEdgesManhattan
 - cppdescent, [12](#)
- CPP-Descent, [1](#)
- cppdescent, [11](#)
 - compareEdgesEuclidean, [12](#)
 - compareEdgesManhattan, [12](#)
 - euclideanDistance, [12](#)
 - KNNBruteForceGraph, [13](#)
 - manhattanDistance, [13](#)
 - NNDescent_KNNGraph, [13](#)
 - NNDescent_Query, [14](#)
 - readBinData, [14](#)
 - readBinGraph, [15](#)
 - recall, [15](#)
 - writeBinGraph, [16](#)
- cppdescent.cpp, [42](#)
 - getSets, [43](#)
 - sampleGraph, [43](#)
- cppdescent.hpp, [38](#)
- euclideanDistance
 - cppdescent, [12](#)
- find
 - PQueue, [20](#)
 - Vector, [25](#)
- findNode
 - Vector, [25](#)
- findPos
 - Vector, [26](#)
- first
 - Vector, [26](#)
- getAt
 - Vector, [26](#)
- getGeneralNeighborsV
 - Graph, [17](#)
- getMax
 - PQueue, [21](#)
- getMin
 - PQueue, [21](#)
- getSets
 - cppdescent.cpp, [43](#)
- getSize
 - PQueue, [21](#)
 - Vector, [27](#)
- getValue
 - vectorNode, [30](#)
- Graph, [17](#)
 - getGeneralNeighborsV, [17](#)
 - insertVertex, [18](#)
- GraphVertex, [18](#)
- GraphVertexPair, [19](#)
- insert
 - PQueue, [21](#)
- insertLast
 - Vector, [27](#)
- insertVertex
 - Graph, [18](#)
- KNNBruteForceGraph
 - cppdescent, [13](#)
- last
 - Vector, [27](#)
- manhattanDistance
 - cppdescent, [13](#)
- next
 - Vector, [27](#)
- NNDescent_KNNGraph
 - cppdescent, [13](#)
- NNDescent_Query
 - cppdescent, [14](#)
- nodeValue
 - Vector, [28](#)
- PQueue, [19](#)
 - ~PQueue, [20](#)
 - find, [20](#)
 - getMax, [21](#)
 - getMin, [21](#)

- getSize, [21](#)
- insert, [21](#)
- PQueue, [20](#)
- remove, [21](#)
- removeMax, [22](#)
- setDestroyValue, [22](#)
- toVector, [22](#)
- previous
 - Vector, [28](#)
- readBinData
 - cppdescent, [14](#)
- readBinGraph
 - cppdescent, [15](#)
- recall
 - cppdescent, [15](#)
- remove
 - PQueue, [21](#)
- removeLast
 - Vector, [28](#)
- removeMax
 - PQueue, [22](#)
- sampleGraph
 - cppdescent.cpp, [43](#)
- setAt
 - Vector, [28](#)
- setDestroyValue
 - PQueue, [22](#)
 - Vector, [29](#)
- sets, [23](#)
- setValue
 - vectorNode, [30](#)
- toVector
 - PQueue, [22](#)
- Vector, [23](#)
 - ~Vector, [24](#)
 - binaryFind, [25](#)
 - find, [25](#)
 - findNode, [25](#)
 - findPos, [26](#)
 - first, [26](#)
 - getAt, [26](#)
 - getSize, [27](#)
 - insertLast, [27](#)
 - last, [27](#)
 - next, [27](#)
 - nodeValue, [28](#)
 - previous, [28](#)
 - removeLast, [28](#)
 - setAt, [28](#)
 - setDestroyValue, [29](#)
 - Vector, [24](#)
- vectorNode, [29](#)
 - getValue, [30](#)
 - setValue, [30](#)
 - vectorNode, [30](#)
- writeBinGraph
 - cppdescent, [16](#)